

Módulo 3: Programación Concurrente

Universidad de las Américas Puebla

Departamento de Actuaría, Física y Matemáticas

Licenciatura en Ciencia de Datos

Profesor: Dr. Víctor Giovanni Morales Murillo

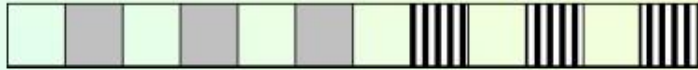
Otoño 2026



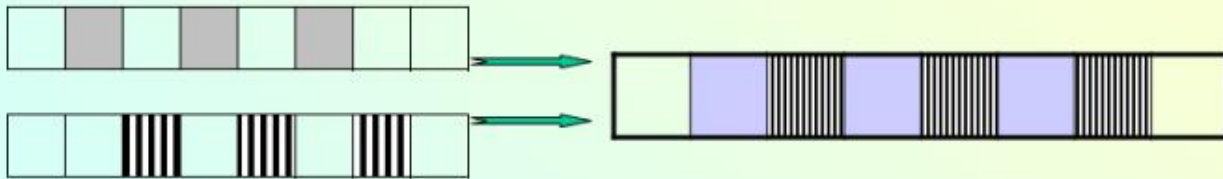
3.1 Antecedentes

Módulo 3: Programación
Concurrente

Monoprogramación



Multiprogramación



Programa 1



Programa 2



Procesador ocioso

Origen de la programación concurrente

- Los primeros programas se ejecutaban de manera **secuencial**: una instrucción después de otra y un programa a la vez.
- Sin embargo, durante la ejecución, el procesador podía permanecer inactivo mientras el programa esperaba operaciones lentas, como:
 - Lectura o escritura de archivos.
 - Entrada de datos del usuario.
 - Acceso a dispositivos.
 - Comunicación por red.
- Para aprovechar mejor el procesador surgió la **multiprogramación**, que permitió mantener varios programas en memoria y alternar su ejecución.

Etapa	Característica principal
1. Sistemas secuenciales	Un programa se ejecuta de principio a fin.
2. Multiprogramación	Varios programas permanecen en memoria.
3. Tiempo compartido	El procesador alterna rápidamente entre procesos.
4. Sistemas multiprocesador	Varias unidades de procesamiento ejecutan actividades simultáneamente.
5. Sistemas multinúcleo	Un procesador integra varios núcleos capaces de trabajar en paralelo.
6. Sistemas distribuidos	Diferentes computadoras colaboran mediante una red.

Evolución hacia la concurrencia

La programación concurrente evolucionó junto con los sistemas de cómputo:

Surgimiento de nuevos problemas

La concurrencia permitió mejorar el aprovechamiento de los recursos y la capacidad de respuesta de los sistemas, pero también introdujo nuevos desafíos.

- Exclusión mutua.
- Sincronización.
- Semáforos y monitores.
- Locks o bloqueos.
- Comunicación entre procesos.

Cuando varios procesos o hilos se ejecutan concurrentemente, pueden:

- Compartir variables, archivos u otros recursos.
- Acceder al mismo dato en momentos diferentes.
- Interferir entre sí.
- Obtener resultados distintos según el orden de ejecución.

Ejemplo de desafío

- Dos hilos incrementan una variable compartida
- El resultado esperado era 7, pero se obtuvo 6.
- Esta situación se denomina **condición de carrera**.
- **Condición de carrera**: ocurre cuando dos o más procesos o hilos acceden concurrentemente a datos compartidos y el resultado depende del orden en que se ejecutan las instrucciones.
- La **programación concurrente** requiere controlar cuidadosamente el **acceso compartido** y el **orden de ejecución**.

Dos hilos incrementan una variable compartida

contador = 5

Hilo A lee contador: 5

Hilo B lee contador: 5

Hilo A escribe: 6

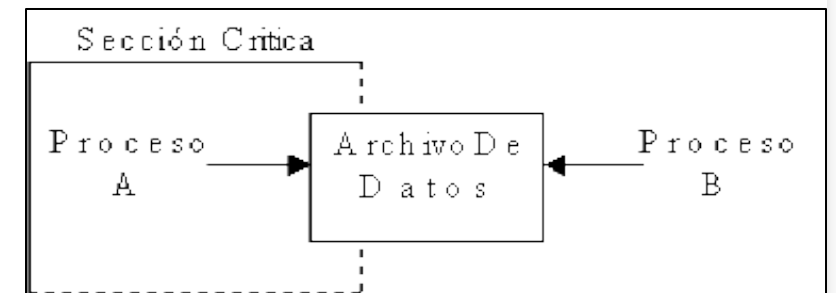
Hilo B escribe: 6

3.2 Algoritmos de exclusión mutua

Módulo 3: Programación
Concurrente

Sección crítica

- El segmento de código en el que un proceso puede modificar variables compartidas con otros procesos o hilos se denomina **sección crítica**.
- Cuando un proceso ejecuta su **sección crítica**, **ningún otro proceso debe acceder simultáneamente a su sección crítica** para evitar inconsistencias.
- El **problema de la sección crítica** consiste en diseñar algún tipo de solución para garantizar que los procesos involucrados puedan operar sin generar ningún tipo de inconsistencia.



Estructura de Código

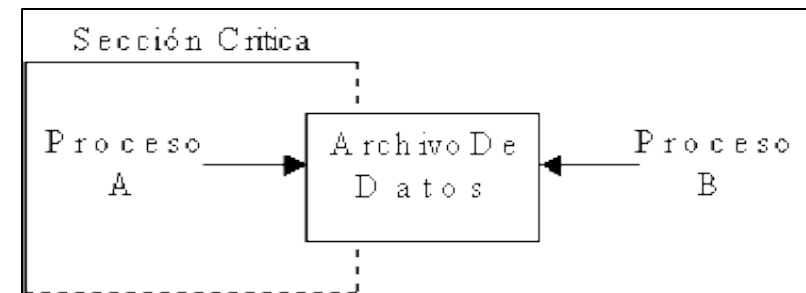
- **Sección de entrada**, en la que se solicita el acceso a la sección crítica.
- **Sección crítica**, en la que se realiza la modificación efectiva de los datos compartidos.
- **Sección de salida**, en la que típicamente se hará explícita la salida de la sección crítica.
- **Sección restante**, que comprende el resto del código fuente.



Requisito	Significado sencillo
Exclusión mutua	Solo un proceso puede estar en la sección crítica a la vez.
Progreso	Si la sección crítica está libre, la decisión de quién entra no puede posponerse indefinidamente.
Espera limitada	Un proceso que solicita entrar no debe esperar indefinidamente mientras otros siguen entrando.

Un algoritmo no es correcto simplemente porque evite que dos procesos entren simultáneamente.

Requisitos de una solución



A. Solución con una variable cerrojo

```
1 int cerrojo = 0;
2
3 do {
4     // SECCIÓN DE ENTRADA
5     while (cerrojo == 1); // No hacer nada.
6     cerrojo = 1;
7     // SECCIÓN CRÍTICA.
8     // ...
9     // SECCIÓN DE SALIDA.
10    cerrojo = 0;
11    // SECCIÓN RESTANTE.
12 }while (1);
```

- Se utiliza una variable compartida cerrojo:
- cerrojo = 0 → sección crítica **libre**.
- cerrojo = 1 → sección crítica **ocupada**.
- **Problema:** comprobar y modificar cerrojo son **dos operaciones separadas**.

A. Solución con una variable cerrojo

Paso	Proceso	cerrojo	Acción	Resultado
1	P0	0	while(cerrojo == 1)	0 == 1 → falso, puede continuar
2	P0	0	Cambio de contexto antes de cerrojo = 1	P0 es interrumpido
3	P1	0	while(cerrojo == 1)	0 == 1 → falso, puede continuar
4	P1	0	cerrojo = 1	P1 bloquea el acceso
5	P1	1	Entra a sección crítica	P1 está dentro
6	P1	1	Cambio de contexto	Regresa P0
7	P0	1	Continúa justo después del while	No vuelve a comprobar el cerrojo
8	P0	1	cerrojo = 1	Sigue valiendo 1
9	P0	1	Entra a sección crítica	P0 también está dentro

```
1 int cerrojo = 0;
2
3 do {
4     // SECCIÓN DE ENTRADA
5     while (cerrojo == 1); // No hacer nada.
6     cerrojo = 1;
7     // SECCIÓN CRÍTICA.
8     // ...
9     // SECCIÓN DE SALIDA.
10    cerrojo = 0;
11    // SECCIÓN RESTANTE.
12 }while (1);
```

Un cambio de contexto entre ambas puede permitir que dos procesos entren a la sección crítica.

B. Alternancia estricta entre dos procesos

```
1 do {
2   // SECCIÓN DE ENTRADA
3   while (turn != i); // No hacer nada.
4   // SECCIÓN CRÍTICA.
5   // ...
6   // SECCIÓN DE SALIDA.
7   turn = j;
8   // SECCIÓN RESTANTE.
9 }while (1);
```

- Se utiliza una variable compartida `turn` para indicar **qué proceso puede entrar a la sección crítica**.
- Si `turn == i`, el proceso **Pi puede entrar**.
- Al salir, **Pi cede el turno a Pj** asignando `turn = j`.
- **Problema:** los procesos están obligados a **alternarse**, aunque uno de ellos no quiera entrar a la sección crítica.

B. Alternancia entre procesos

Paso	Proceso	turn	Acción	Resultado
1	P0	0	while(turn != 0)	Falso, P0 puede entrar
2	P0	0	Entra a sección crítica	P0 está dentro
3	P1	0	while(turn != 1)	Verdadero, P1 espera
4	P0	0	Sale de sección crítica	P0 termina
5	P0	0	turn = 1	Cede el turno a P1
6	P1	1	while(turn != 1)	Falso, P1 puede entrar
7	P1	1	Entra a sección crítica	P1 está dentro
8	P0	1	Intenta volver a entrar	1 != 0 → verdadero, P0 espera
9	P1	1	Sale de sección crítica	Termina
10	P1	1	turn = 0	Cede el turno a P0
11	P1	0	Quiere volver a entrar inmediatamente	0 != 1 → verdadero, P1 no puede entrar
12	P0	0	Permanece en sección restante	P0 no quiere entrar
13	P1	0	Sigue esperando	P1 queda bloqueado

```
do {  
    while (turn != i);  
  
    // SECCIÓN CRÍTICA  
  
    turn = j;  
  
    // SECCIÓN RESTANTE  
  
} while (true);
```

- **Cumple exclusión mutua** : nunca entran los dos al mismo tiempo.
- **No cumple progreso**: un proceso puede quedar esperando a otro que ni siquiera quiere entrar.

C. Arreglo de banderas

```
1 do {
2     // SECCIÓN DE ENTRADA
3     flag[i] = true;
4     while (flag[j]); // No hacer nada.
5     // SECCIÓN CRÍTICA.
6     // ...
7     // SECCIÓN DE SALIDA.
8     flag[i] = false;
9     // SECCIÓN RESTANTE.
10 }while (1);
```

- Se utiliza un arreglo booleano compartido flag para indicar si cada proceso **quiere entrar a la sección crítica**.
- flag[i] = true → **Pi quiere entrar**.
- flag[i] = false → **Pi no quiere entrar**.
- Antes de entrar, Pi comprueba si Pj también quiere entrar.

C. Arreglo de banderas

Paso	Proceso	flag[0]	flag[1]	Acción	Resultado
1	P0	F	F	flag[0] = true	P0 indica que quiere entrar
2	P0	T	F	Cambio de contexto	P0 es interrumpido
3	P1	T	F	flag[1] = true	P1 indica que quiere entrar
4	P1	T	T	while(flag[0])	true → P1 espera
5	P0	T	T	Continúa su ejecución	—
6	P0	T	T	while(flag[1])	true → P0 espera
7	—	T	T	Ambos esperan	Ninguno puede entrar

```
1 do {
2     // SECCIÓN DE ENTRADA
3     flag[i] = true;
4     while (flag[j]); // No hacer nada.
5     // SECCIÓN CRÍTICA.
6     // ...
7     // SECCIÓN DE SALIDA.
8     flag[i] = false;
9     // SECCIÓN RESTANTE.
10 }while (1);
```

- **Exclusión mutua:** los dos procesos **no entran simultáneamente** a la sección crítica.
- **No cumple progreso:** si ambos colocan su bandera en true antes de comprobar la bandera del otro

D. Arreglo de banderas con orden diferente

Paso	Proceso	flag[0]	flag[1]	Acción	Resultado
1	P0	F	F	while(flag[1])	false → P0 puede continuar
2	P0	F	F	Cambio de contexto	P0 aún no ejecuta flag[0]=true
3	P1	F	F	while(flag[0])	false → P1 puede continuar
4	P1	F	T	flag[1] = true	P1 indica que entra
5	P1	F	T	Sección crítica	P1 entra
6	P1	F	T	Cambio de contexto	Regresa P0
7	P0	T	T	flag[0] = true	P0 continúa después del while
8	P0	T	T	Sección crítica	P0 también entra

```

1 do {
2     // SECCIÓN DE ENTRADA
3     while (flag[j]); // No hacer nada.
4     flag[i] = true;
5     // SECCIÓN CRÍTICA.
6     // SECCIÓN DE SALIDA.
7     flag[i] = false;
8     // SECCIÓN RESTANTE.
9 }while (1);

```

- No cumple exclusión mutua.

E. Solución de Peterson (2 Procesos)

```
1 do {
2   // SECCIÓN DE ENTRADA
3   flag[i] = true;
4   turn = j;
5   while (flag[j] && turn == j); // No hacer nada.
6   // SECCIÓN CRÍTICA.
7   // ...
8   // SECCIÓN DE SALIDA.
9   flag[i] = false;
10  // SECCIÓN RESTANTE.
11 }while (1);
```

- El **algoritmo de Peterson** proporciona una solución de exclusión mutua para **dos procesos**.
- Combina dos ideas anteriores:
 - flag[i]: indica que el proceso **P_i** **quiere entrar** a la sección crítica.
 - turn: indica **qué proceso tiene prioridad** cuando ambos quieren entrar.

E. Solución de Peterson (2 Procesos)

Paso	Proceso	flag[0]	flag[1]	turn	Acción
1	P0	T	F	—	flag[0] = true
2	P0	T	F	1	turn = 1
3	—	T	F	1	Cambio de contexto
4	P1	T	T	1	flag[1] = true
5	P1	T	T	0	turn = 0
6	P1	T	T	0	flag[0] && turn==0 → true → espera
7	P0	T	T	0	flag[1] && turn==1 → false → entra
8	P0	F	T	0	Salida: flag[0] = false
9	P1	F	T	0	flag[0] && turn==0 → false → entra
10	P1	F	F	0	Salida: flag[1] = false

```
1 do {
2   // SECCIÓN DE ENTRADA
3   flag[i] = true;
4   turn = j;
5   while (flag[j] && turn == j); // No hacer nada.
6   // SECCIÓN CRÍTICA.
7   // ...
8   // SECCIÓN DE SALIDA.
9   flag[i] = false;
10  // SECCIÓN RESTANTE.
11 }while (1);
```

- El **algoritmo de Peterson** combina flag y turn para garantizar exclusión **mutua**, **progreso** y **espera limitada** entre **dos procesos**.

¿Qué ocurre cuando
tenemos 3, 10 o 100
procesos?

Algoritmo de la panadería de Lamport (N Procesos)



Listado 1.18: Solución n procesos; algoritmo de Lamport

```
1 do {
2   // SECCIÓN DE ENTRADA
3   eleccion[i] = true;
4   num[i] = max(num[0], ..., num[n]) + 1;
5   eleccion[i] = false;
6   for (j = 0; j < n; j++) {
7     while (eleccion[j]);
8     while (num[j] != 0 && (num[j], j) < (num[i], i));
9   }
10  // SECCIÓN CRÍTICA.
11  // ...
12  // SECCIÓN DE SALIDA.
13  num[i] = 0;
14  // SECCIÓN RESTANTE.
15 }while (1);
```

Algoritmo de la panadería de Lamport

Paso	Proceso	Acción	eleccion	num	Resultado
1	P0	eleccion[0] = true	[T,F,F]	[0,0,0]	P0 empieza a elegir turno
2	P0	num[0] = max(...) + 1	[T,F,F]	[1,0,0]	P0 toma turno 1
3	P0	eleccion[0] = false	[F,F,F]	[1,0,0]	P0 termina de elegir
4	P1	eleccion[1] = true	[F,T,F]	[1,0,0]	P1 empieza a elegir
5	P1	num[1] = max(...) + 1	[F,T,F]	[1,2,0]	P1 toma turno 2
6	P1	eleccion[1] = false	[F,F,F]	[1,2,0]	P1 termina de elegir
7	P2	eleccion[2] = true	[F,F,T]	[1,2,0]	P2 empieza a elegir
8	P2	num[2] = max(...) + 1	[F,F,T]	[1,2,3]	P2 toma turno 3
9	P2	eleccion[2] = false	[F,F,F]	[1,2,3]	P2 termina de elegir
10	P2	while(elección[0])	[F,F,F]	[1,2,3]	Ya terminaron de elegir
11	P2	while(num[0]!=0 && (num[0,0) < (num[2],2))	[F,F,F]	[1,2,3]	True && true ->P2 Espera

```

1 do {
2     // SECCIÓN DE ENTRADA
3     eleccion[i] = true;
4     num[i] = max(num[0], ..., num[n]) + 1;
5     eleccion[i] = false;
6     for (j = 0; j < n; j++) {
7         while (eleccion[j]);
8         while (num[j] != 0 && (num[j], j) < (num[i], i));
9     }
10    // SECCIÓN CRÍTICA.
11    // ...
12    // SECCIÓN DE SALIDA.
13    num[i] = 0;
14    // SECCIÓN RESTANTE.
15 }while (1);

```

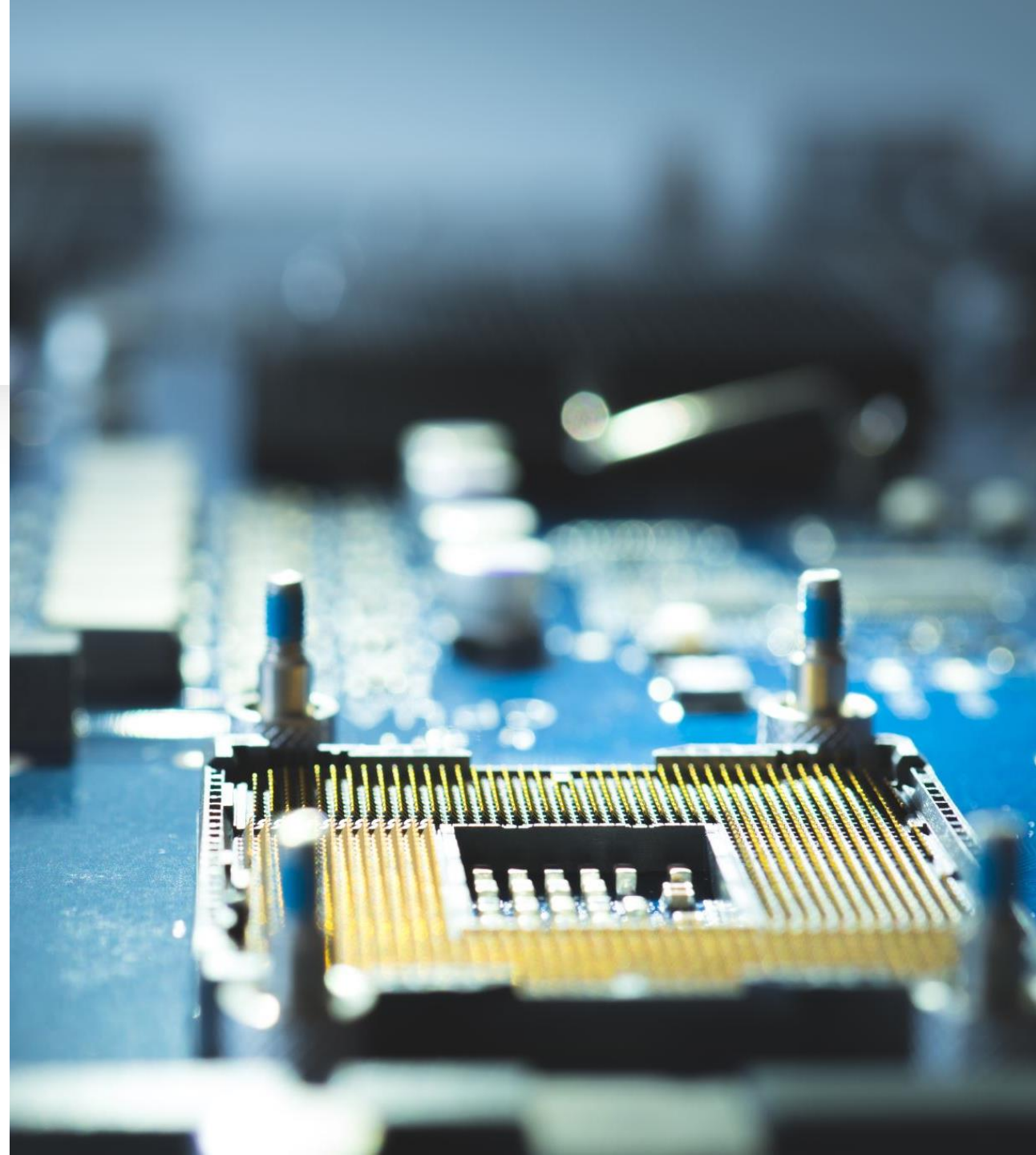
El **algoritmo de Lamport** garantiza **exclusión mutua** porque cada proceso espera mientras exista otro con un turno menor que el suyo.

Primer while: espera a que el otro saque su boleto.

Segundo while: si su boleto va antes que el mío, espero.

Solución hardware: Deshabilitar Interrupciones

- En un sistema con **un solo CPU**, un proceso puede deshabilitar las interrupciones antes de entrar a su **sección crítica**.
- Al deshabilitar las interrupciones, se evita que el proceso sea desalojado por una interrupción de reloj durante la ejecución de la sección crítica.
- **Limitaciones:**
 - Funciona principalmente en sistemas con **un solo CPU**.
 - En sistemas multiprocesador, otro CPU puede continuar ejecutando procesos.
 - Deshabilitar las interrupciones de todos los CPU sería costoso.
 - Puede afectar funciones del sistema que dependen de interrupciones.
 - Un error dentro de la sección crítica, como un ciclo infinito, podría provocar problemas graves.



Instrucciones con mayor atomicidad

- Algunos procesadores proporcionan instrucciones capaces de realizar **lectura y modificación de memoria como una única operación atómica**.
- Una de ellas es **Test-and-Set**.
- Obtiene el valor actual de una variable y, al mismo tiempo, la reemplaza por un nuevo valor de manera **atómica**.

```
public class TestAndSet {  
  
    private int valor = 0;  
  
    public synchronized int testAndSet(int nuevoValor) {  
  
        int valorAnterior = valor;  
        valor = nuevoValor;  
        return valorAnterior;  
    }  
}
```

Aquí synchronized **simula la atomicidad para fines didácticos**; no significa que Java esté ejecutando literalmente la instrucción hardware test-and-set.

Exclusión mutua con Test-and-Set

- **TestAndSet** convierte conceptualmente las operaciones de comprobar y modificar del cerrojo en una operación atómica.
- **Problema: espera activa (busy waiting)** y consume CPU.
- **Swap** es otra instrucción de hardware, que permite **intercambiar atómicamente** los valores de dos ubicaciones de memoria y puede utilizarse para construir mecanismos de **exclusión mutua**.

Paso	Proceso	lockFlag antes	TestAndSet(1) devuelve	lockFlag	Resultado
1	P0	0	0	1	P0 entra
2	P1	1	1	1	P1 espera
3	P1	1	1	1	P1 sigue esperando
4	P0	1	TestAndSet(0)	0	P0 sale
5	P1	0	0	1	P1 entra
6	P1	1	TestAndSet(0)	0	P1 sale

```
public void requestCS(int i) {  
    while (lockFlag.testAndSet(1) == 1);  
}  
  
public void releaseCS(int i) {  
    lockFlag.testAndSet(0);  
}
```

3.3 Sincronización de procesos y hebras de control

Módulo 3: Programación
Concurrente

Sincronización de procesos e hilos

- La **sincronización** consiste en **coordinar la ejecución de procesos o hilos** para garantizar que determinadas operaciones ocurran en el **momento y orden adecuados**.
- Es necesaria cuando:
 - Existen **recursos o datos compartidos**.
 - Una tarea **depende del resultado** de otra.
 - Un **proceso o hilo** debe **esperar un evento o la finalización** de otro.

Exclusión mutua y sincronización

- La exclusión mutua es una forma de sincronización.
- Pero la sincronización también permite establecer un orden entre las operaciones de diferentes procesos o hilos.

Exclusión mutua

Evita accesos simultáneos

Protege la sección crítica

¿Quién puede acceder?

Sincronización

Coordina la ejecución

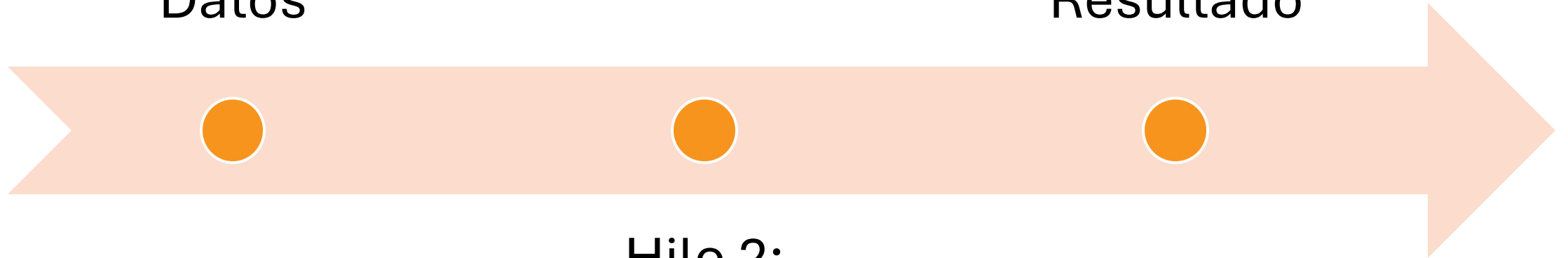
Establece orden o dependencias

¿Cuándo puede continuar?

Ejemplo de sincronización

Hilo 1: Obtener
Datos

Hilo 3: Mostrar
Resultado



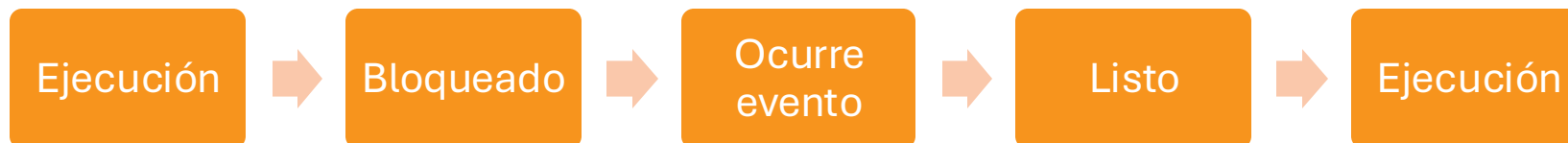
Hilo 2:
Procesar Datos

Espera activa y espera bloqueada

- **Espera activa**
- El proceso o hilo comprueba repetidamente una condición hasta que pueda continuar.



- **Espera bloqueada**
- El proceso o hilo suspende su ejecución mientras espera que ocurra un evento o se cumpla una condición.



Sincronización en Java

- En Java, un hilo puede crearse implementando la interfaz Runnable y asociándolo a un objeto Thread.
- start() inicia un **nuevo hilo de ejecución** y hace que la JVM invoque el método run() de la tarea.

Interfaz Runnable	Creación de hilo
<pre>class Tarea implements Runnable { @Override public void run() { System.out.println("Ejecutando tarea"); } }</pre>	<pre>Thread hilo = new Thread(new Tarea()); hilo.start();</pre>

Sincronización en Java

join() hace que el hilo que ejecuta esa instrucción **espere hasta que hilo termine**.

MÉTODO JOIN

```
Thread hilo = new Thread(new Tarea());
```

```
hilo.start(); // Inicia el hilo  
hilo.join();  // Espera a que termine
```

```
System.out.println("La tarea terminó");
```

